PANDUAN DASAR • LINUX UBUNTU

Pengenalan Bash Scripting

Otomatiskan pekerjaan harian di terminal Ubuntu — dari perintah sederhana hingga script backup.

halo.sh

```
#!/bin/bash

echo "Halo, Bash!"

$ ./halo.sh
Halo, Bash!
```

**materi dibuat dengan bantuan AI, disesuaikan oleh Muhammad Mishbah*

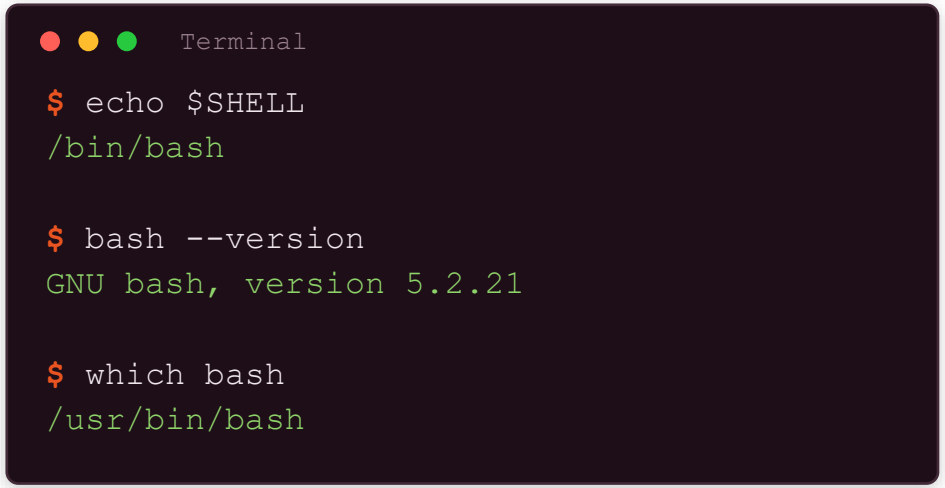
Apa Itu Bash?

Bash (Bourne Again SHell) adalah shell default di Ubuntu: program penerjemah yang mengubah perintah yang kita ketik menjadi aksi pada sistem operasi.

1. **Shell bawaan**
   Terpasang otomatis di setiap instalasi Ubuntu, tanpa perlu software tambahan.

2. **Dua cara pakai**
   Interaktif (ketik satu per satu) atau lewat script (dijalankan sekaligus).

3. **Script = file .sh**
   Sekumpulan perintah terminal yang disimpan dalam satu berkas teks.



Cek shell yang sedang Anda pakai dengan perintah di atas.

Perintah dasar untuk latihan

`pwd` `ls -l` `cd` `cat` `mkdir` `rm`

Semua perintah ini bisa dipakai di dalam script.

Mengapa Belajar Bash Scripting?

Empat alasan utama script Bash masih jadi andalan pengguna Linux.

1 Otomatisasi

Tugas berulang seperti backup, update sistem, dan pembersihan file cukup dijalankan dengan satu perintah.

2 Hemat Waktu

Puluhan langkah manual diringkas menjadi satu berkas yang bisa dipakai ulang kapan saja.

3 Konsisten

Langkah yang sama dieksekusi persis sama setiap kali, sehingga risiko salah ketik jauh berkurang.

4 Ringan & Bawaan

Tidak butuh instalasi atau bahasa tambahan — Bash sudah tersedia di semua sistem Ubuntu.

Struktur Dasar & Cara Menjalankan

hello.sh

```
#!/bin/bash
# Baris di atas disebut shebang
# Komentar diawali tanda pagar

echo "Selamat datang di Bash"
date
echo "Direktori: $PWD"
```

Shebang `#!/bin/bash` memberi tahu sistem bahwa berkas ini harus dijalankan menggunakan Bash. Selalu tulis di baris pertama.

Hasil

```
$ ./hello.sh
Selamat datang di Bash
Sel 01 Sep 2026 09:12:33 WIB
Direktori: /home/andi
```

Tiga langkah menjalankan script

1. **Buat berkas**

```
$ nano hello.sh
```

2. **Beri izin eksekusi**

```
$ chmod +x hello.sh
```

3. **Jalankan script**

```
$ ./hello.sh
```

Variabel & Input Pengguna

variabel.sh

```
#!/bin/bash

# Membuat variabel (tanpa spasi di tanda =)
nama="Andi"
umur=20
echo "Halo $nama, umur $umur tahun"

# Meminta input dari pengguna
read -p "Masukkan kota: " kota
echo "Kamu tinggal di $kota"

# Variabel bawaan sistem
echo "User aktif : $USER"
echo "Folder ini : $PWD"
```

Aturan penting

- Tidak ada spasi di sekitar tanda sama dengan.
- Panggil isi variabel dengan awalan $nama.
- Bungkus dengan kutip ganda agar aman dari spasi.
- Nama variabel peka huruf besar-kecil.

Output

```
$ ./variabel.sh
Halo Andi, umur 20 tahun
Masukkan kota: Jakarta
Kamu tinggal di Jakarta
```

Konvensi umum gunakan huruf kecil untuk variabel biasa, dan HURUF KAPITAL untuk nilai tetap seperti SUMBER atau TUJUAN.

Percabangan: if, elif, else

nilai.sh

```
#!/bin/bash

read -p "Masukkan nilai: " nilai

if [ $nilai -ge 75 ]; then
  echo "Selamat, kamu lulus!"
elif [ $nilai -ge 60 ]; then
  echo "Kamu harus remedial."
else
  echo "Maaf, belum lulus."
fi

# Mengecek keberadaan file
if [ -f "catatan.txt" ]; then
  echo "File ditemukan"
fi
```

Beri spasi di dalam kurung siku: [$a -eq $b]

Operator perbandingan

`-eq`	sama dengan
`-ne`	tidak sama dengan
`-gt`	lebih besar dari
`-lt`	lebih kecil dari
`-ge`	lebih besar atau sama
`-le`	lebih kecil atau sama
`-f`	berkas tersebut ada
`-d`	folder tersebut ada

Perulangan: for & while

for — jumlah pengulangan sudah diketahui

for.sh

```
#!/bin/bash

for i in 1 2 3 4 5; do
  echo "Perulangan ke-$i"
done

# Menelusuri semua file .txt
for file in *.txt; do
  echo "Ditemukan: $file"
done
```

while — diulang selama syarat terpenuhi

while.sh

```
#!/bin/bash

angka=1

while [ $angka -le 5 ]; do
  echo "Angka: $angka"
  angka=$((angka + 1))
done

echo "Perulangan selesai"
```

Catatan Gunakan `$((...))` untuk operasi hitung, `break` untuk keluar dari perulangan, dan `continue` untuk melompat ke putaran berikutnya.

Fungsi & Argumen Script

fungsi.sh

```
#!/bin/bash

# Mendefinisikan fungsi
sapa() {
  echo "Halo, $1! Umur $2 tahun."
}

# Memanggil fungsi
sapa "Budi" 20

# Argumen yang dikirim dari terminal
echo "Nama script  : $0"
echo "Argumen ke-1 : $1"
echo "Jumlah       : $#"
```

Tulis fungsi sebelum dipanggil — Bash membaca script dari atas ke bawah.

Variabel khusus

`$0`	Nama berkas script
`$1 $2`	Argumen pertama, kedua
`$#`	Jumlah argumen
`$@`	Semua argumen
`$?`	Status perintah terakhir

Cara memanggil

```
$ ./fungsi.sh Ani 21
```

Contoh Praktis: Script Backup Otomatis

backup.sh

```
#!/bin/bash
# Mencadangkan folder Dokumen ke folder backup

SUMBER="$HOME/Dokumen"
TUJUAN="$HOME/backup"
TANGGAL=$(date +%Y-%m-%d)
NAMA="backup-$TANGGAL.tar.gz"

mkdir -p "$TUJUAN"

if [ -d "$SUMBER" ]; then
  tar -czf "$TUJUAN/$NAMA" "$SUMBER"
  echo "Backup berhasil: $TUJUAN/$NAMA"
else
  echo "Folder sumber tidak ditemukan!"
  exit 1
fi
```

Yang terjadi di script ini

`$(date ...)`

Menjalankan perintah lain dan menyimpan hasilnya ke variabel.

`mkdir -p`

Membuat folder tujuan bila belum ada, tanpa pesan error.

`tar -czf`

Memampatkan seluruh folder menjadi satu arsip .tar.gz.

`exit 1`

Menghentikan script dengan penanda bahwa terjadi kesalahan.

*Ingin berjalan otomatis setiap hari? Jadwalkan dengan **`crontab -e`**.*

Tips & Langkah Selanjutnya

1 Biasakan rapi

Awali dengan shebang, beri komentar, dan gunakan indentasi agar script mudah dibaca.

2 Amankan variabel

Selalu bungkus dengan kutip ganda seperti "$file" agar nama berspasi tidak error.

3 Uji sebelum dipakai

Jalankan bash -x script.sh untuk melihat setiap langkah yang dieksekusi.

4 Lanjut belajar

Pelajari array, case, sed, awk, serta penjadwalan tugas dengan cron.

```
$ echo "Mulai dari script kecil, otomatiskan satu tugas harian Anda."
```

Terima kasih